VUE.JS

UTILISATION D'UN FRAMEWORK

Chapitre essentiellement repris de https://vuejs.org/guide

VUE.JS

Pourquoi un framework ?

Le HTML / CSS / JavaScript respecte des standards évoluants peu.

Le développement d'applications web a besoin de techniques / logiques modernes et modulables.

Pensez à l'ex. 6 (sliders) et imaginez créer une application web comme Discord, YouTube, Spotify, un géoportail, etc

Un framework est un ensemble de fonctions / modules / outils pour faciliter le développement.

Cela simplifie les tâches fastidieuses et organise mieux le code.

VUE.JS

Installation

Vue.js peut être utilisé comme librairie simple (fichier JS a inclure dans votre page), ou comme module de NodeJS.

Dans les exercices, l'installation se fera avec NodeJS.

VUE.JS

Première application

L'application Vue.js de base est :

```
<div id="app"></div>

<script type="module">
import { createApp } from 'vue'

const app = createApp(/* élément racine de l'application */)
app.mount('#app')
</script>
```

L'élément `<div>` est un élément HTML div standard

Le code JavaScript est placé dans une balise script qui est un module, comme d'habitude

VUE.JS

Première application

L'application Vue.js de base est :

```
<div id="app"></div>

<script type="module">
import { createApp } from 'vue'

const app = createApp(/* élément racine de l'application */)
app.mount('#app')
</script>
```

La fonction `createApp` est la fonction de base de Vue. Elle prend en paramètre l'élément racine de l'application.
La fonction `mount` de l'application crée l'application (il ne se passe rien avant). Elle prend en paramètre l'élément HTML dans lequel créer l'application, ici l'élément avec l'id "app".

VUE.JS

Première application

L'application Vue.js de base est :

```
<div id="app"></div>

<script type="module">
import { createApp } from 'vue'
import App from './App.vue'

const app = createApp(App)
app.mount('#app')
</script>
```

Généralement, on déclare l'élément racine dans un fichier séparé, qu'on inclus.

Le contenu de la balise `script` est souvent aussi placé dans un fichier "main.js"

VUE.JS

Première application

Les composants de Vue sont généralement placés dans des fichiers .vue

```
// Fichier App.vue

<script></script>

<template>Contenu du composant</template>

<style></style>
```

On trouve trois éléments dans la définition d'un composant

1. Une balise `script` qui définit la logique du composant et son initialisation
2. Une balise `template` qui définit le code HTML du composant
3. Une balise `style` qui définit le style propre au composant

VUE.JS

Première application

```
<script>
  export default {
    data() { // Déclaration de l'état du composant et de ses propriétés
      return { count: 0 }
    },
    methods: { // Méthodes propres au composant
      increment() { this.count++ }
    },
    mounted() { /* script d'initialisation */ }
  }
</script>
<template>Contenu du composant</template>
<style></style>
```

Dans la balise script, on peut trouver plusieurs définitions dont

- `data()` qui retourne un objet contenant les propriétés du composant
- `methods` qui retourne un objet contenant les méthodes du composant
- `mounted()` qui est une fonction exécutée quand le composant est "monté" (ajouté au HTML de la page)

VUE.JS

Première application

```
<script>
  export default {
    data() {
      return { count: 0 }
    },
    methods: {
      increment() { this.count++ }
    },
    mounted() { }
  }
</script>
<template>
  <p>{{count}}</p>
  <button v-on:click="increment">Incrémenter</button>
</template>
<style></style>
```

On peut utiliser des propriétés dans le HTML avec la syntaxe `{{ prop }}`

`v-on` est ce qu'on appelle une *directive*, c'est un élément spécifique à Vue qui décrit un comportement particulier, ici une méthode à appeler au clic sur le bouton.

VUE.JS

Première application

```
<script>
  export default {
    data() {
      return { count: 0 }
    },
    methods: {
      increment() { this.count++ }
    },
    mounted() { }
  }
</script>
<template>
  <p>{{count}}</p>
  <button v-on:click="increment">Incrémenter</button>
</template>
<style></style>
```

Au centre de Vue se trouve le principe de *réactivité*, ici `count` est une propriété qui, quand elle est modifiée, va automatiquement mettre à jour tous les éléments qui en dépendent.

Cliquer sur le bouton va incrémenter `count` et mettre à jour le contenu de la balise `p`, automatiquement.

VUE.JS

Première application

```
<script>
  export default {
    data() {
      return { displayed: true }
    },
    methods: {
      toggleDisplay() { this.displayed=!this.displayed }
    },
    mounted() { }
  }
</script>
<template>
  <p v-if="displayed" >Paragraphe affiché</p>
  <button @click="toggleDisplay">Afficher / Cacher le paragraphe</button>
</template>
<style></style>
```

La directive `v-if` permet d'indiquer si un élément doit être affiché ou non

La directive `v-on` peut être remplacée par `@`, qui est plus court

VUE.JS

Les *computed properties*

```
<script>
  export default {
    data() {
      return { age: 5 }
    },
    computed: {
      isAdult() { return this.age >= 18}
    },
    methods: { },
    mounted() { }
  }
</script>
<template>
  <p>Age : {{age}}, {{isAdult ? 'Majeur' : 'Mineur'}}</p>
  <button @click="this.age+=1">+</button>
  <button @click="this.age-=1">-</button>
</template>
<style></style>
```

Dans `computed` on peut indiquer des propriétés qui dépendent d'autres.

Dans des `{{ ... }}` il est possible de mettre des ternaires (`condition` ? `siVrai` : `siFaux`)

VUE.JS

Les *computed properties*

```
<script>
  export default {
    data() {
      return { age: 5 }
    },
    computed: {
      isAdult() { return this.age >= 18}
    },
    methods: { },
    mounted() { }
  }
</script>
<template>
  <p>Age : {{age}}, {{isAdult ? 'Majeur' : 'Mineur'}}</p>
  <button @click="this.age+=1">+</button>
  <button @click="this.age-=1">-</button>
</template>
<style></style>
```

La différence entre les *computed properties* et les méthodes est que la valeur des *computed properties* est mise en cache. Elle n'est recalculée que si une propriété dont dépend la *computed property* change.

VUE.JS

Les directives : v-bind

La directive `v-bind` permet d'assigner une valeur à une propriété HTML

```
<div v-bind:id="dynamicId"></div>
```

Elle peut être abrégée par `":"`

```
<div :id="dynamicId"></div>
```

```
<button :disabled="isButtonDisabled">Button</button>
```

VUE.JS

Les directives : v-bind

Pour les classes, il est possible d'indiquer par un boolean l'activation ou non de la propriété.

```
<div :class="{ active: isActive }"></div>
```

Ici la `div` n'aura la classe "active" que si `isActive` est vrai

Pour les styles, le même principe s'applique pour en définir des propriétés

```
<div :style="{ color: activeColor, fontSize: fontSize + 'px' }"></div>
```

Des opérations JavaScript peut être faites dans les directives, ici une addition

VUE.JS

Les directives : v-if

v-if ne monte l'élément que si sa condition est vrai

```
<h1 v-if="isAwesome">Vue is awesome!</h1>
```

Il est possible également d'avoir des `v-if`, `v-else-if` et `v-else`

```
<div v-if="type === 'A'">A</div>
<div v-else-if="type === 'B'">B</div>
<div v-else-if="type === 'C'">C</div>
<div v-else>Not A/B/C</div>
```

Si un `v-if` est faux, l'élément ne sera pas ajouté au code de la page, n'existera pas du point de vue du HTML, et aucun de ses événements ne sera traité

VUE.JS

Les directives : v-show

v-show affiche l'élément si la condition est vrai

```
<h1 v-show="isDisplayed">Hello!</h1>
```

Contrairement à `v-if`, `v-show` ne fait que cacher l'élément, mais celui-ci continue d'exister dans le code de l'application.

VUE.JS

Les directives : v-for

`v-for` génère du HTML pour chaque élément d'une liste

```
data() {
  return {
    items: [{ message: 'Foo' }, { message: 'Bar' }]
  }
}
```

```
<li v-for="item in items">{{ item.message }}</li>
```

Cela générera le code suivant

```
<li>Foo</li>
<li>Bar</li>
```

Cette directive est particulièrement utile quand on souhaite générer du HTML pour un nombre variable d'éléments, non connus à l'avance.

VUE.JS

Les directives : v-on

`v-on` permet d'exécuter du code lors d'événements

```
<button v-on:click="onButtonClicked">Greet</button>
```

Cette directive peut-être abrégée par "@"

```
<button @click="onButtonClicked">Greet</button>
```

VUE.JS

Template refs

Les références de balises dans la section `template` se font via l'attribut `ref`

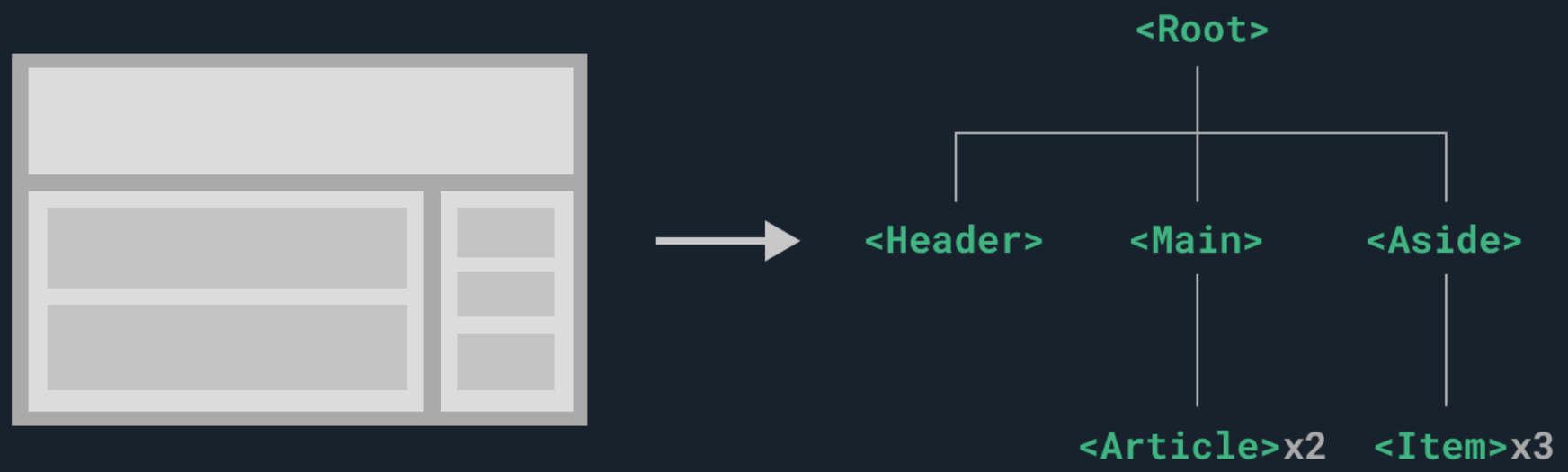
```
<script>
  export default {
    mounted() {
      this.$refs.input.focus() // Utilisation de l'input via les références
    }
  }
</script>

<template>
  <input ref="input" />
</template>
```

Cela est utile quand le code JS a besoin d'accéder à une balise et qu'il n'existe pas de directive appropriée.

VUE.JS

Les composants

Les composants sont les constituants d'une application Vue.



Jusqu'à maintenant nous avons travaillé dans un composant uniquement

VUE.JS

Les composants

```
<script>
  import ButtonCounter from './ButtonCounter.vue'

  export default {
    components: {
      ButtonCounter
    }
  }
</script>

<template>
  <h1>Voici des ButtonCounter : </h1>
  <ButtonCounter />
  <ButtonCounter />
</template>
```

VUE.JS

Les composants - Props

Il est possible de créer des propriétés dans ses composants :

```
// Dans le composant parent
<MonComposant user-message="hello" admin-message="salut" />
```

Cela est fait de la manière suivante :

```
// Dans MonComposant.vue
export default {
  props: ['userMessage', 'adminMessage'], // Liste de propriétés, sous forme de tableau
  created() {
    console.log(this.userMessage) // props are exposed on `this`
  }
}
```

Cela est utile pour passer une valeur d'un composant parent à un composant enfant

A noter que la propriété est utilisée dans le template avec le *kebab-case* `user-message` mais avec le *camelCase* `userMessage` dans le code. Vue fait automatiquement la conversion.

VUE.JS

Les composants - Props

Ces propriétés peuvent être utilisées avec les directives

```
// Dans le composant parent
<MonComposant :user-message="this.messageForUser" :admin-message="this.messageForAdmin" />
```

Les composants fonctionnent comme des éléments HTML standards

Attention, les propriétés passées à un composant sont en lecture seule

```
export default {
  props: ['userMessage', 'adminMessage'],
  created() {
    this.userMessage = 'monMessage' // Interdit !
  }
}
```

Ceci afin d'éviter qu'un composant enfant ne modifie l'état de son parent (effet de bord). Si on doit modifier la valeur d'une propriété, on peut soit créer une propriété interne à l'objet, ou utiliser une *computed property*.

VUE.JS

Les composants - Événements

Comment un composant enfant peut-il signaler un changement a un composant parent ?

VUE.JS

Les composants - Événements

Le composant enfant peut envoyer des événements à l'aide de `$emit`

Cela peut être fait dans la partie template

```
<button @click="$emit('someEvent')">Click Me</button>
```

Ou dans la partie script

```
export default {
  methods: {
    submit() {
      this.$emit('someEvent')
    }
  }
}
```

VUE.JS

Les composants - Événements

Le composant parent écoute un événement à l'aide de son nom

```
<childComponent @some-event="callback" />
```

Où `callback` est une méthode du composant parent.

A noter que l'élément émit est écrit en camelCase (`someEvent`) et l'attribut est noté en kebab-case (`some-event`). Vue fait automatiquement la conversion.

VUE.JS

Les composants - Événements

Les événements peuvent avoir des arguments

```
<button @click="$emit('increaseBy', 1)">Increase by 1</button>
```

Et les arguments sont passés à la fonction de l'objet parent

```
<MyButton @increase-by="(n) => count += n" />
```

VUE.JS

Conclusion

Quelques informations supplémentaires :

- Vue possède deux syntaxes principales, dites *Options* et *Composition*. Nous avons vu la syntaxe *Options*. Vous pourriez lire une syntaxe différente sur internet.
- Vue a beaucoup évolué ces dernières années. Nous avons vu la syntaxe de *Vue 3*. Certaines ressources sur internet pourraient utiliser d'anciennes versions.
- Nous n'avons fait qu'un survol des principaux concepts de Vue, il y a beaucoup de sujet que nous n'avons pas vu. N'hésitez pas à lire la documentation officielle : https://vuejs.org/guide